LIBXS Samples

Bracket Units

This sample fits piecewise-constant targets with networks of truncated-power units $<w*x - b>^m$, and measures how sharp the learned edges are. The 1-D targets are a step (`step`), a square wave (`square`), and the step with Gaussian noise of sigma = 0.1 on the training samples (`noisy`). The 2-D targets are a checkerboard (`checker`), the same checkerboard turned by 30° (`turned`, see `BRACKET_ANGLE`), and a disc (`disc`). Training uses Adam with full batches; the two-dimensional models use LIBXS GEMM. By default the training data are pixels, i.e., averages over cells, as an image sensor records them.

Variant	Model
`direct`	One hidden layer of $<w*x - b>^1$ (ReLU) units fitted to the samples.
`integrated`	The same model fitted to the running integral, then differentiated.
`*_wd`	As above, with decoupled weight decay.
`integrated_merge`	Integrated, then jumps within one sample spacing merged and refined.
`integrated_greedy`	As merged, then jumps inserted where the residual calls for one.
`integrated_prune`	As greedy, then jumps removed while the fit stays within the estimated noise.
`mlp`	Two hidden ReLU layers fitted to the pixels.
`mlp_int`	The same network fitted to the integral along (1,1), then d/dx + d/dy.
`tensor`	Products $<x - a>^1<y - b>^1$ fitted to the pixels.
`tensor_int`	The same products fitted to the summed-area table, then $d^2/dxdy$.
`sep`	Sums of products A(x)*B(y) of 1-D bracket fits, fitted to the pixels.
`sep_int`	The same fitted to the summed-area table (merged, greedy), then $d^2/dxdy$.
`rot`	One product A(u)*B(v) in a turned frame (u, v), fitted to the pixels.
`rot_int`	The same fitted to running sums in that frame, then $d^2/dudv$; the angle is searched.
`box`	Products of clamped ramps $(<x - a>^1 - <x - b>^1)(<y - e>^1 - <y - g>^1)$, fitted to the pixels.
`box_int`	The same fitted to the summed-area table, then $d^2/dxdy$: a sum of rectangles.
`box_greedy`	Rectangles added one at a time where the residual gains most, over a background; refined on the pixels.
`knn`	libxs_predict on the class (sign) of each pixel: nearest neighbours.
`knn_score`	As `knn`, in the label-aware score coordinate.
`fisher`	As `knn`, with Fisher-weighted inputs.
`hknn`	As `knn`, over a kd-partition.
`rf`	A random forest on the class of each pixel.
`knn_avg`	Nearest neighbours regressing the pixel values (pixel data only).
`xgb`	XGBoost on the class of each pixel, if built with it.

The variants up to `integrated_prune` run on each 1-D target, the others on each 2-D target.

Build and Run

```
cd samples/bracket
make
./bracket.x
```

The binary takes no arguments; the environment variables below configure it. The two-dimensional networks dominate the runtime. The sample links BLAS by default; `make BLAS=0` falls back to the built-in GEMM, which is slower. XGBoost is found as in the predict sample (`XGBOOST_ROOT`), and `make XGB=0` builds without it. With XGBoost enabled (the default), the build overrides `PEDANTIC` to 0.

Each row of the printed table reports:

Column	Meaning
`err`	Error against the noise-free target on the fine evaluation grid.
`pos/h`	Mean distance between true and learned edges, in training-grid spacings.
`slope`	Growth of the order-1 fingerprint norm under refinement: 0.5 is a jump, 0 a smooth edge.

Column	Meaning
`eps/h`	Effective edge width implied by that norm, in training-grid spacings.
`fpdist`	Fingerprint distance (`libxs_fprint_diff`, orders 0 and 1) to the noise-free target.
`time`	Time to fit the variant, in seconds.

For the 2-D targets, the fingerprint norms are the root mean square over image rows.

Each classifier from `knn` to `xgb` also prints the share of the evaluation grid it answers at confidence 0.9 and the precision there, the precision of its most confident 50% and 80%, and, for libxs_predict, its number of coordinates (`ncoord`).

Environment Variables

Variable	Default	Meaning
`BRACKET_OUTDIR`	.	Directory for MHD and PNG output (empty: no output).
`BRACKET_NSTEPS`	20000	Adam steps in 1-D; the 2-D models run a quarter of them.
`BRACKET_WD`	0.01	Weight decay of the `*_wd` variants.
`BRACKET_SEED`	1	Seed for initialization and noise.
`BRACKET_UNITS`	32	Hidden units of the 1-D models (at most 64).
`BRACKET_SAMPLES`	64	Training samples of the 1-D targets (4 to 256).
`BRACKET_WIDTH`	64	Hidden width of the 2-D networks (at most 1024).
`BRACKET_2D`	1	Set to 0 to skip the 2-D targets.
`BRACKET_PIXEL`	1	Set to 0 for point samples instead of cell averages.
`BRACKET_RANK`	4	Largest number of products in `sep` and `sep_int`.
`BRACKET_ANGLE`	30	Angle of the turned checkerboard in degrees.

`bracket_1d_<target>.mhd` stacks the target and each variant's output as rows. `bracket_2d_<target>.mhd` holds a target (`checker`, `turned`, or `disc`), and `bracket_2d_<target>_<variant>.mhd` each model's image, clamped to the target's range [-1, 1] after the metrics are taken. The variants from `knn` on also write their confidence as `bracket_2d_<target>_<variant>_conf.mhd`. The directory must exist. Each MHD file has a PNG beside it: the two-dimensional images on [-1, 1] and the confidence on [0.5, 1], so that images compare side by side, and a 1-D stack over its own range.

Converse Sample

Converse reads plain text or Markdown files and then answers questions about them, summarizes them, assembles new text from them, and trains small next-token models on them. Answers are extracted from the ingested text and carry a citation, or the sample abstains; nothing is generated from outside the corpus unless you ask for it.

It is built only on libxs primitives (lexemes, metatokens, fingerprints, registries, n-grams, parameter prediction) and is self-contained: nothing is downloaded, no external service is contacted, and no data is used beyond the files you give it. It is experimental and adds no stable public API.

Build

```
cd samples/converse
make PEDANTIC=2
```

This builds `converse.x`, `converse-qa.x`, `converse-lm.x`, `converse-norm.x` and `summarize.x`. Without a built library the sample compiles header-only; with one (`make PEDANTIC=2` in the `libxs` root first), it links it. From the `libxs` root, `make PEDANTIC=2 samples/converse/` does the same.

Which binary

binary	serves	use it for
`converse-qa.x`	interactive, `-e`, `-c`, `-L`	questions, evaluation, recombination
`converse-lm.x`	`-E`, `-c -K KIND`, `-L`	next-token models, byte model
`converse-norm.x`	prompt, `-e`, `-t`, `-r`, `-m`, `-L`	spelling normalization
`converse.x`	everything above but normalization	anything, and all documented examples

`converse.x` accepts every command below. The split binaries link only what they serve, and a mode a binary does not serve is rejected up front, naming the one that does. The recombination probe fills its bpc columns only under `converse.x` with `CONVERSE_HIER_RESCORE=1`; otherwise it prints `no seam judge` and leaves them at zero.

Run any binary without arguments to print its options and the main environment variables.

Getting text in

Put files anywhere; `texts/` is the conventional place and is ignored by git. Two structures are understood:

- **prose** — paragraphs of running text, blank-line separated. Uppercase lines are read as section headings and become the citation for everything under them.
- **Markdown** — headings, paragraphs and code blocks; `#` headings become the citation.

The structure is chosen by file extension (`.md` is Markdown, everything else is prose) and can be forced with `-p prose|markdown`.

Pass files directly, or use `-b PREFIX` to pick up a numbered set without listing it:

```
./converse.x texts/prose1.txt texts/prose2.txt
./converse.x -b texts/grimm                    # texts/grimm.txt, grimm-1.txt, ...
./converse.x -b docs ../../documentation/*.md
```

`-b PREFIX` probes `PREFIX`, `PREFIX.txt` and numbered parts (`PREFIX-N.txt`, `PREFIX_N.txt`, `PREFIX.N.txt`); missing siblings are fine as long as one file is usable. Markdown files are not probed, so pass them explicitly (a glob is fine, as above).

`-b` also names the **state files**: the basename of the prefix is the prefix of everything the run persists, so `-b docs` keeps `docs.dat`, `docs.lex` and so on, and several corpora can live side by side in one directory. When `-b` is given more than once, the first one names the state. Without `-b`, state is named after the current directory (`converse.*` when run from the sample directory).

Any UTF-8 text works. Suitable sources are Project Gutenberg books, a Wikipedia extract, your own notes, or the documentation of this repository. Larger corpora are supported; memory is the practical limit.

Warm and cold runs State is reused: a second run over the same files loads the persisted corpus, lexicon and reranker instead of re-ingesting, which is much faster. To force a full rebuild, delete the state files for that prefix:

```
rm -f grimm.dat grimm.par grimm.src grimm.lex grimm.prd grimm.facts
```

Do this before comparing two runs — a warm run answers from state built by the previous one.

To build the state without asking anything — useful before a batch of runs, or to time ingestion on its own — use `-L`:

```
./converse.x -L -b texts/grimm
```

Asking questions

```
./converse.x -b texts/grimm
```

The sample then reads one question per line:

```
> Who is Gretel?
Gretel is the girl.
citation: HANSEL AND GRETEL (texts/grimm.txt:2544)
> Where did Hans go?
Hans went into the stable.
citation: CLEVER HANS (texts/grimm.txt:6031)
> Who was eaten by the wolf?
Grandmother and Little Red-Cap were eaten by the wolf.
citation: LITTLE RED-CAP [LITTLE RED RIDING HOOD] (texts/grimm.txt:3104-3112)
> What belongs to Curdken?
hat belongs to Curdken.
citation: THE GOOSE-GIRL (texts/grimm.txt:1771)
> What do we know about Hansel?
Hansel is the boy. Hansel stood still and peeped back at the house.
citation: HANSEL AND GRETEL (texts/grimm.txt:2544-2579)
> Who is Sherlock Holmes?
I do not know from the corpus.
```

What is answerable depends on the rule files (next section). With the shipped rules the sample answers:

- `Who is X?` — an identity or a definition ("X is a Y", "X, a Y, ...").
- `Where did X go?` / `Where is X?` — a place, as a proposition from X's own sentence.
- `Who was V by X?` — a relation, including passives it was never told about.
- `What belongs to X?` — an enumeration, each item cited.
- `What do we know about X?` — several cited propositions collected about one name.
- `How are X and Y connected?` / `What connects X and Y?` — the path between two entities, stated as the propositions it is made of.
- anything else question-shaped — the best matching sentence, ranked and cited.

Every answer is followed by a citation naming where it came from: the section title when the corpus has headings, and always the file and the line. An answer assembled from several propositions cites the range of lines it rests on (`texts/grimm.txt:2544-2579`), one range per file, and a single line when it is one line. Titles are only printed when the text really carries them — a corpus of flat prose is cited by file and line alone rather than by a sentence that happened to look like a heading.

A connection question is answered by stating the facts that join the two entities, each with its own citation, and never by inventing a relation between them:

```
> How are McClellan and Douglas connected?
Lincoln restored McClellan. Lincoln forced Douglas.
citation: texts/wiki2m.txt:813-895
```

If nothing in the corpus joins them, the sample says so rather than answering with a sentence about one of them.

Three behaviours to expect:

- The sample **abstains** (`I do not know from the corpus.`) rather than guessing when the corpus does not support an answer.
- A question whose KIND it does not recognize — one using none of the declared `ask|` words — still gets an answer, introduced by `I did not recognize the question. The closest the corpus comes:` so a relevant sentence is never mistaken for a direct answer. That is different from abstaining: the question was not read, rather than read and unsupported.
- A question of the form `In TITLE, ...` is ranked only against the matching heading, which is how to ask about one document in a corpus of many.

Questions are case-insensitive.

Non-question input is treated as a composition request and answered from the fingerprint path instead. `-n N` sets how many sentences a response may use (default 1).

Checking answers against a fixture

`-e` runs a fixture of questions and expected terms and prints a pass count, which is how to tell whether a rule-file or corpus change helped:

```
./converse.x -e -b texts/grimm
```

The fixture is `<prefix>.eval` (so `grimm.eval` for `-b grimm`). Each non-comment line has three required fields and two optional ones:

```
question|evidence-terms|reply-terms|fact-terms|citation-terms
```

The fifth field checks the citation, and it is matched against the whole citation — title, file and line — so a fixture can assert `texts/grimm.txt:2544`.

Terms are comma-separated and matched case-insensitively. An empty evidence field marks an abstention case (the question SHOULD go unanswered); an empty reply field skips the concise-reply check. The fact field is checked only when relation facts were learned, so one fixture works with and without rule files.

`-P PROFILE` selects the profile used to rank candidate answers (`raw`, `poly2`, `smooth`, `temporal`, `rf`, `fisher`, `hknn`), for both interactive use and `-e`:

```
./converse.x -P temporal -e texts/prose1.txt
```

Normalizing spellings

`converse-norm.x` reads the corpus vocabulary as a codebook and maps a surface form to the nearest word in it, so whatever it answers is a word the corpus contains. With no mode it prompts, one word per line, and prints the decision together with the words it chose between:

```
./converse-norm.x -b texts/grimm

> rabbet
rabbet -> rabbit  (distance 1)
    rabbit (261)
> quene
quene: abstained, nearest at 2 with 4 tied
    queen (373)
    quite (242)
```

`-r` sets the largest edit distance a correction may span (default 1) and `-m` how far the runner-up must be beyond the chosen word (default 1, which accepts everything). `-e` reports how densely the vocabulary packs and how often a corrupted word is recovered, which is how to pick the two:

```
./converse-norm.x -e -r 1 -m 2 -b texts/grimm
```

A wrong correction silently replaces the word that was typed: a larger `-r` recovers more but corrects wrongly more often, and a larger `-m` abstains more often. Run `-e` on your own corpus before trusting a setting.

`-t` reads surface forms from a file, one per line, and writes the accepted corrections to `<prefix>.norms`:

```
printf 'rabbet\nqueeen\n' > forms.txt
./converse-norm.x -t forms.txt -b texts/grimm
```

Every binary loads that table on its next run and applies it while encoding, so a question asked with a corrected spelling answers as the corrected one would. Forms that decode to themselves, and those with no word in reach, are left out.

Summarizing and composing

`summarize.x` is a separate, smaller entry point:

```
./summarize.x texts/prose1.txt              # fuse adjacent sentences
./summarize.x -n 3 texts/prose1.txt         # to a target sentence count
printf 'First sentence. Second one.' | ./summarize.x -
./summarize.x -g -r -n 8 texts/prose1.txt texts/prose2.txt   # compose
```

- default: summarize one file by repeatedly fusing adjacent sentences. A single argument that is not a readable file, or several arguments, are summarized as literal text (joined by spaces); no argument or `-` reads stdin.
- `-n N`: target sentence count; without it, fusion continues while a pair can be fused and each fusion is printed.
- `-g`: compose mode — ingest all files, take the first as the target, and emit a short assembled text (`-n` is the phrase budget, default 12). The ingested corpus is kept in `summarize.dat` in the working directory, and the next run loads and extends it.
- `-r`: reflow the input first, joining cosmetic line breaks (useful for Gutenberg files); in compose mode only the target is reflowed.
- `-h` (or `--help`) as the first argument: print the usage.

Progress and diagnostics go to stderr, the result to stdout.

Recombining text from the corpus

An experimental mode splices clauses from different sentences into new ones, keeping each half verbatim and cutting only at clause boundaries:

```
CONVERSE_RECOMB=50 CONVERSE_HIER_RESCORE=1 ./converse.x -E -x -b texts/grimm
```

`CONVERSE_RECOMB=N` attempts N joins (50 when N is not positive) and reports how many were made, with coherence and seam statistics. `CONVERSE_HIER_RESCORE=1` trains the hierarchical byte model (`-K hier`) and uses it to score each seam (the bpc columns) and to reorder candidate answers; it takes effect only in the binaries that link that model (`converse.x`, `converse-lm.x`). `CONVERSE_RECOMB_COMPOSE=1` additionally lets the interactive `-c` mode answer with a spliced sentence. It is off by default, since it emits text that does not occur verbatim in the corpus.

Next-token prediction

An experimental predictor is trained from the token stream and kept out of the question-answering path:

```
./converse.x -E -b texts/grimm              # accuracy, default trigram
./converse.x -E -K bigram -b texts/grimm    # pick the model
./converse.x -E -H 10 -b texts/grimm        # held-out: train on 9/10, test on 1/10
printf 'the little\n' | ./converse.x -c -b texts/grimm   # suggestions + continuation
CONVERSE_GRAN=meta-word ./converse.x -E -b texts/grimm   # token unit
```

- `-E` reports accuracy, `-c` prints the top few successors plus a short continuation.
- `-H N` evaluates on a held-out split, `-T PREFIX` on a separate corpus, which is what to use for an honest, non-memorized figure.
- `-K` selects the model: `bigram`, `trigram` (default), `predict`, `embed`, `rerank`, `knnlm`, or `hier` (evaluation-only: metatoken, byte and PPM bits per character).
- `-P PROFILE` selects the predictor profile for `predict|embed|rerank` (`raw`, `poly2`, `smooth`, `temporal`, `rf`, `fisher`, `hknn`).
- `-x` uses the deepest n-gram context.
- `CONVERSE_GRAN` sets the token unit: `word`, `native`, `syllable`, `bpe`, `meta-native`, `meta-word`, `meta-syllable`.

An optional `<prefix>.predict` fixture of `context|expected-next` lines adds a curated check to `-E`. Many further knobs exist; run the binary with no arguments to list them.

Teaching it a language and a corpus

Everything corpus- and language-specific lives in local rule files rather than in the source, and they are what decide which questions can be answered. Two files, both optional:

- `converse.rules` — the LANGUAGE: function words and syntactic classes. The English file is part of the repository. A `<prefix>.rules` file REPLACES it (for a corpus in another language).
- `<prefix>.relations` — this CORPUS: aliases, role words, place names. It EXTENDS the language file, so it need not restate function words.

Each non-comment line is `kind|term`:

kind	declares	example
`alias`	a query verb and the verb the text uses	`alias|eaten|devoured`
`person`	a role word that can be bound to a name	`person|grandmother`
`place`	a place noun	`place|forest`
`where`, `why`, `how`	the markers those questions turn on	`where|in`
`topic`	the marker of an attribute question	`topic|about`
`own`	the verb a possession question uses	`own|belongs`
`poss`	how the language WRITES possession	`poss|apostrophe-s`
`copula`, `article`, `prep`	the syntactic classes definitions need	`copula|is`
`aux`	the auxiliaries	`aux|had`
`agent`	the word a passive names its agent with	`agent|by`
`genitive`	the word that marks a possessor after the thing	`genitive|of`
`join`	how the language writes a joiner inside a name	`join|ampersand`
`link`	the words that ask how two entities relate	`link|connected`
`ask`	a question KIND and this language's word for it	`ask|who|who`
`pron`	the back-reference pronouns a follow-up points with	`pron|it`
`result`	the light verb introducing a resulting state	`result|made`
`skip`, `negate`	filler words and negators	`skip|the`

A few notes that matter in use:

- `where|in` plus `place|forest` is what makes `Where ...?` answerable; declaring only one of the two answers nothing.
- `aux|had` and `agent|by` are enough to read passives and active clauses generally — the sample derives which words the corpus uses as verbs and as nouns from those frames and reports `verbs derived: N` and `nouns derived: N`. No verb list is needed.
- `poss|apostrophe-s` (English) against `poss|apostrophe` (German): declare the wrong one and possession answers go silent rather than wrong.
- `ask|` is how the sample knows a question's KIND. The first field is the kind (`who`, `what`, `where`, `when`, `why`, `how`, `yesno`), the second is the word — so a German rule file writes `ask|who|wer`. A question using none of the declared words still gets an answer, prefixed with a note that the question was not recognized.
- `person|father` plus `genitive|of` is what makes kinship answerable in BOTH forms — "Lincoln's father Thomas" and "Aegeus, the father of Theseus". A role word you do not declare is simply not read, so extend the `person|` class for the kinds of relation your text states.

The sample can also PROPOSE rules instead of only reading them: `CONVERSE_RULES_LEARN=N` offers up to N `person|` class members found in the corpus, which is a way to bootstrap a rule file for new text. What it proposes is speculative, so it is reported separately and, if a `<prefix>.learn.eval` fixture exists, that fixture is used for `-e` while learning is on.

After ingestion the sample reports what it learned (`relation facts: N learned`, `identity facts`, `location facts`, `type facts`). Set `CONVERSE_FACTS_LIST=1` to print the facts themselves, plus a `graph reach:` line (edges, entities, pairs joined by one middle, largest degree). Few edges means connection questions will mostly go unanswered on that corpus.

Optional `<prefix>.bridges` adds evidence-backed answer frames, five fields per line:

```
name|query-groups|evidence-groups|score|reply
```

Whitespace separates required groups and `/` separates alternatives; `_` stands for a literal space in evidence terms. The reply is literal text or a small frame such as `{after:lighthouse had}`.

Environment variables

The variables used above, all unset by default:

variable	effect
`CONVERSE_PREDICT=1`	train the answer reranker and save it as `<prefix>.prd` (skipped otherwise)
`CONVERSE_GRAN=UNIT`	token unit for next-token models (`word` by default)
`CONVERSE_NGRAM_ORDER=N`	n-gram context order 1–6 (default 2; `-x` selects 6)
`CONVERSE_RECOMB=N`	run the recombination probe over N joins
`CONVERSE_RECOMB_COMPOSE=1`	let `-c` answer with a spliced sentence
`CONVERSE_HIER_RESCORE=1`	score seams and reorder answers with the hierarchical model
`CONVERSE_RULES_LEARN=N`	propose up to N `person|` class members as rules
`CONVERSE_FACTS_LIST=1`	print the learned facts and the `graph reach:` line

Running a binary without arguments lists these and the further tuning variables of the next-token models.

Files the sample writes

All state is kept in the working directory, named after the `-b` prefix (after the directory without one), and all of it except `.norms` is ignored by git:

file	holds
`<prefix>.dat`	the ingested corpus
`<prefix>.par`	the source texts the corpus refers to; required with `.dat`
`<prefix>.lex`	lexicon and token IDs
`<prefix>.prd`	the answer reranker, trained only with `CONVERSE_PREDICT=1`
`<prefix>.src`	the file names the corpus refers to, for citations
`<prefix>.facts`	the derived fact layers, rebuilt when the corpus changes
`<prefix>.norms`	spelling corrections, written by `converse-norm.x -t`

Delete one of `.dat` / `.par` and both are rebuilt. The fixtures and rule files (`<prefix>.eval`, `<prefix>.learn.eval`, `<prefix>.predict`, `converse.rules`, `<prefix>.relations`, `<prefix>.bridges`) are inputs you write; the sample never modifies them.

Fingerprint

This sample exercises the LIBXS fingerprint API (`libxs_fprint`) and reports its properties as CSV tables. Rosetta covers the higher-level algebra and type discovery.

What It Produces

Set `FPRINT_OUTDIR` to collect CSV artifacts (without it, results go to stdout only):

```
mkdir -p results/fprint
FPRINT_OUTDIR=results/fprint ./fprint.x
```

The sample writes:

File	Purpose
`convergence.csv`	Same functions sampled at increasing resolution and compared to a high-resolution reference.
`sensitivity.csv`	Perturbations with matched element RMS but different structural character.
`geometry.csv`	Bracket-polynomial boundary area, centroid, moments, and shape fingerprints.
`hierarchy.csv`	2-D smooth, creased, diagonal-creased, and noisy fields in hierarchical and per-axis modes.

File	Purpose
`compression.csv`	Newton truncation order versus reconstruction error.
`collision.csv`	Pseudometric collision test: pairs with shared L2 norms but different signed means, and a pair sharing the whole fingerprint; `bernstein_distance` is `libxs_fprint_bernstein_diff`.
`timing.csv`	Wall-clock cost per element at various input sizes.
`convergence_rate.csv`	Log-log convergence rate (slope and R-squared) for smooth functions.

Build

```
cd samples/fprint
make
```

Batched GEMM

Exercises the LIBXS batched GEMM API (`libxs_gemm.h`) in several flavors: strided, pointer-array, indexed, and grouped. All programs are multithreaded via OpenMP, report wall-clock time and GFLOPS/s, and optionally print a checksum of the results via `libxs_matdiff`.

Building

```
cd samples/gemm
make
```

The C programs build header-only when LIBXS is not built yet; only `gemm_indexf.x` (Fortran) needs LIBXS built first from the repository root. BLAS is linked dynamically and sequential by default (`BLAS=1`, `BLAS_STATIC=0`). `make MKL=1` (sequential) or `make MKL=2` (threaded) links MKL and enables MKL's JIT kernels; `make LIBXSMM=1` enables LIBXSMM kernels and needs a built sibling `libxsmm` directory.

Programs

gemm_strided Strided batch DGEMM: all matrices packed contiguously with constant stride. Uses `libxs_gemm_index` with index_stride=0.

```
./gemm_strided.x [M [N [K [batchsize [nrepeat [beta [pad]]]]]]]
```

Defaults: M=N=K=23, batchsize=30000, nrepeat=3, beta=1.0, pad=0.

gemm_batch Pointer-array batch DGEMM: matrices accessed through pointer arrays. Supports a duplicate C-matrix mode to exercise lock-forward sync.

```
./gemm_batch.x [M [N [K [batchsize [nrepeat [dup [beta [pad]]]]]]]]
```

Defaults: M=N=K=23, batchsize=30000, nrepeat=3, dup=0, beta=1.0, pad=0.

dup	Mode	Description
0	None	Each C-matrix is unique (LIBXS_GEMM_FLAG_NOLOCK)
1	Sorted	Half-unique C-pointers, duplicates consecutive
2	Shuffled	Half-unique C-pointers, randomly shuffled

gemm_index Index-array batch DGEMM: matrices in contiguous buffers, addressed through explicit element-offset index arrays (ia, ib, ic).

```
./gemm_index.x [M [N [K [batchsize [nrepeat [beta [pad]]]]]]]
```

Defaults: M=N=K=23, batchsize=30000, nrepeat=3, beta=1.0, pad=0.

gemm_indexf Fortran variant of gemm_index. Demonstrates the LIBXS Fortran module interface with one-based index arrays and C_LOC/C_SIZEOF interop. Requires a Fortran compiler and a built LIBXS (skipped otherwise).

Compiled with OpenMP, `libxs_gemm_index` (and `libxs_gemm_batch`) run over an OpenMP team unless called from within a parallel region, sized by `libxs_gemm_ntasks` (printed as `libxs_gemm_index tasks`). `Performance` times this plain call, `Task split` the explicit `libxs_gemm_index_task`.

```
./gemm_indexf.x [M [N [K [batchsize [nrepeat]]]]]
```

Defaults: M=N=K=23, batchsize=30000, nrepeat=3.

gemm_groups Grouped batch DGEMM: multiple groups of different matrix shapes dispatched in sequence, each with its own `libxs_gemm_config_t`. Per-group dimensions grow by 4 starting from base_m.

```
./gemm_groups.x [ngroups [batch_per_group [nrepeat [base_m [beta [pad]]]]]]
```

Defaults: ngroups=2 (max 4), batch_per_group=30000, nrepeat=3, base_m=8, beta=1.0, pad=0.

Validation

Set the CHECK environment variable to any non-zero value to print a checksum of the first and the last C-matrix (`libxs_matdiff`):

```
CHECK=1 ./gemm_strided.x
CHECK=1 ./gemm_batch.x 23 23 23 1000 1 2
CHECK=1 ./gemm_index.x
CHECK=1 ./gemm_groups.x
```

When padding is enabled (pad>0), the check also verifies that leading-dimension padding in C-matrices has not been overwritten.

Runtime Controls

`LIBXS_GEMM_PANEL=N` sets the smallest column panel a split SYRK/SYR2K uses; zero (the default) takes `LIBXS_GEMM_BN`. A larger value yields fewer and wider panels, i.e., fewer tasks. `LIBXS_GEMM_BM`, `LIBXS_GEMM_BN` and `LIBXS_GEMM_BK` size the tiles of a panel's diagonal block when no BLAS SYRK is available.

Set `LIBXS_GEMM_PRINT=0` to print a compact GEMM registry summary at termination. The first line reports registry size, capacity, memory, and the selected `LIBXS_GEMM_BACKEND` policy; the second line reports a histogram by datatype and backend; the third line reports the occupied warm-up slots and how many of them settled without a JIT kernel (`nojit`).

Memory Comparison Benchmark

Benchmarks `libxs_diff`, `libxs_memcmp`, and the C standard library's `memcmp` for both large contiguous comparisons and many small fixed-size comparisons. A Fortran variant compares `libxs_diff` against Fortran array intrinsics.

Building

```
cd samples/memory
make
```

Produces `memcmp.x` and `itrans.x` (C). The Fortran variants `memcmpf.x` and `matcpyf.x` are built only when a Fortran compiler is found (GNU Fortran needs 4.6 or later) and the library was built beforehand: without a built library the sample is compiled header-only and the Fortran variants are skipped.

Usage (C)

```
./memcmp.x [elsize [stride [nelems [niters]]]]
```

Argument	Default	Description
elsize	INSIZE/MAXSIZE (compile, 64)	Element comparison size in bytes
stride	max(MAXSIZE, elsize)	Distance between successive elements
nelems	2 GB / stride	Number of elements
niters	5	Repetitions (arithmetic average)

Environment Variables

Variable	Default	Description
STRIDED	0	0: one-call when stride==elsize, else loop; 1: seq loop; >=2: random
CHECK	0	Non-zero: validation pass (inject diffs, cross-check implementations)
MISMATCH	0	0: equal buffers; 1: first byte; 2: middle; 3: last; >=4: random
OFFSET	0	Shift buffer b off its natural alignment by this many bytes

Compile-Time Knobs

Macro	Default	Description
MAXSIZE	64	Stride floor (elements spaced >= this far)
INSIZE	MAXSIZE	Default element size when argument is omitted

```
## 32-byte elements, sequential strided access, 10 repetitions
./memcmp.x 32 0 0 10

## 8-byte elements, random traversal, 3 repetitions
STRIDED=2 ./memcmp.x 8 0 0 3

## contiguous (single-call) comparison, ~2 GB
./memcmp.x 64 64 0 5

## mismatch at last byte, buffer b misaligned by 3 bytes
MISMATCH=3 OFFSET=3 ./memcmp.x 32 0 0 5
```

Environment variables and compile-time macros apply only to `memcmp.x`.

Usage (C — itrans)

```
./itrans.x [m [n [ldi [ldo]]]]
```

Argument	Default	Description
m	64	First matrix dimension
n	m	Second matrix dimension
ldi	m (enforced m)	Leading dimension of input
ldo	n (enforced n)	Leading dimension of output

Checks the in-place transpose `libxs_itrans` and its parallel variant `libxs_itrans_task` against a reference. It prints only on a mismatch; the exit status tells whether both checks passed. The element type is `double`, selected at compile time with `-DELEM_TYPE=type`.

Usage (Fortran — memcmpf)

```
./memcmpf.x [nelements [nrepeat]]
```

Element type is hard-coded as INTEGER(4). Compares ~2 GB by default, with 5 repetitions. Reports per-iteration times (ms) and average throughput (MB/s).

Usage (Fortran — matcpyf)

```
./matcpyf.x [m [n [ldi [ldo [nrepeat [nmb]]]]]]
```

Argument	Default	Description
m	4096	First matrix dimension
n	m	Second matrix dimension
ldi	m (enforced m)	Leading dimension of input
ldo	ldi (enforced m)	Leading dimension of output
nrepeat	2	Number of repetitions
nmb	2048	Memory budget in MB

Benchmarks `libxs_matcopy` and `libxs_matcopy_task` for matrix copy and zeroing. When the matrix is square and ldi==ldo, also benchmarks `libxs_otrans` and `libxs_otrans_task` for out-of-place transpose.

What Is Measured

Three comparison implementations are benchmarked:

- `libxs_diff` — SIMD-dispatched short-buffer comparison. Limited to elsize < 256 bytes, per-element.
- `libxs_memcmp` — SIMD-dispatched memcmp replacement. Single call for contiguous buffers, or per-element for strided access.
- stdlib `memcmp` — C library implementation under the same pattern.

Each kernel gets freshly initialized data before its timed section. An untimed warm-up pass runs before the first measured iteration. The summary reports min / median / max MB/s across iterations. Reported MB/s counts both buffers (2 * nbytes / time).

Ozaki-Scheme Low-Precision GEMM

Intercepts DGEMM, SGEMM, ZGEMM, and CGEMM at link time, replacing them with low-precision Ozaki schemes (mantissa slicing or CRT). Real GEMM calls go through the selected scheme; complex GEMM (ZGEMM/CGEMM) is mapped to real GEMM internally.

Build

```
cd samples/ozaki
make -j $(nproc)
```

The sample builds header-only when LIBXS is not built yet. When a sibling `libxstream` directory is detected, the optional OpenCL/GPU path is compiled in (see OZAKI_OCL below); `make OCL=0` or `make LIBXSTREAM=0` leaves it out (default: `OCL=2`). BLAS is linked multi-threaded (`BLAS=2`) by default, or sequential (`BLAS=1`) with `make OMP=0`.

`make LIBXSMM=1` puts LIBXSMM's int8 kernels ahead of the built-in VNNI/AMX kernels; `OZAKI_XSMM=0` returns to the built-in kernels at runtime. It needs a built sibling `libxsmm` directory. `make DNNL=1` (with `DNNLROOT` set) runs Scheme-1 int8 GEMMs through oneDNN, and `make TESTROOT=<path>` compiles in external test code (see `TESTROOT.md` in this directory).

Link-Time Interception

One of two link-time variants is built for each of DGEMM, SGEMM, ZGEMM, and CGEMM, depending on how BLAS is linked:

- `dgemm-wrap.x`, `sgemm-wrap.x`, `zgemm-wrap.x`, `cgemm-wrap.x` — static BLAS (default), linked with `--wrap=` (GNU ld)
- `dgemm-blas.x`, `sgemm-blas.x`, `zgemm-blas.x`, `cgemm-blas.x` — `make BLAS_STATIC=0`, dynamically linked against BLAS and intercepted with `LD_PRELOAD=libwrap.so`

Static (`libwrap.a`) and shared (`libwrap.so`) wrapper libraries are built by default. The shared library can intercept any application:

```
LD_PRELOAD=/path/to/libwrap.so ./application
```

The static library requires explicit `--wrap` flags:

```
-Wl,--wrap=dgemm_ -Wl,--wrap=sgemm_ -Wl,--wrap=zgemm_ -Wl,--wrap=cgemm_
```

Test scripts: `test-wrap.sh` exercises all variants, `test-check.sh` validates both schemes, `test-mhd.sh` tests MHD file-based input.

Test Driver

```
dgemm-wrap.x [A.mhd|M [B.mhd|N [K [TA [TB [ALPHA [BETA [LDA [LDB [LDC]]]]]]]]]]
```

Defaults: M=N=K=257, TA=TB=0, ALPHA=BETA=1.0. Passing MHD filenames as the first two arguments loads A and B from files. The `zgemm-*.x` and `cgemm-*.x` drivers call ZGEMM and CGEMM.

Environment Variables

Scheme Selection

Variable	Default	Description
OZAKI	3	0=bypass (BLAS), 1=mantissa slicing, 2=CRT, 3=adaptive
OZAKI_COMPLEX	(auto)	Complex dispatch: 0=BLAS, 1=CPU, 2=GPU+fallback, 3=BLAS, with real GEMMs issued inside it kept on BLAS. Auto: 0 if OZAKI=0, else 2
OZAKI_N	(auto)	Slices (Sch.1: fp64=8, fp32=4) or moduli (Sch.2: fp64=16, fp32=9)
OZAKI_AMX	(auto)	CPU: 0=VNNI, 1=AMX. Auto: AMX for Sch.1, for Sch.2 above a size threshold
OZAKI_XSMM	1	With `make LIBXSMM=1`: 0=built-in int8 kernels, 1=LIBXSMM

OZAKI=3 (the default) selects the scheme automatically. On the CPU, it is Scheme 1 where AMX is available (unless OZAKI_AMX=0), and Scheme 2 otherwise. On the GPU, LIBXSTREAM selects the scheme per call. An explicit OZAKI applies to both.

Threading (CPU)

Variable	Scheme 1 (OZAKI=1)	Scheme 2 (OZAKI=2)
OMP_PROC_BIND	true	true
OMP_PLACES	cores	cores
OMP_NUM_THREADS	number of cores	unset

Keep threads bound as above; unbound threads are slower.

Accuracy

Variable	Default	Description
OZAKI_FLAGS	(auto)	Sch.1: 3=triangular and symmetrized, 0=full S^2 square (other values are ignored). Auto: 3 below OZAKI_SYM_XOVER, else 0
OZAKI_SYM_XOVER	1048	Largest of M, N, K from which auto OZAKI_FLAGS selects 0
OZAKI_TRIM	0	Levels to trim (0=default); positive values lose accuracy, negative values add precision
OZAKI_MAXK	32768	Max K per preprocessing pass (0=full K in one pass)
OZAKI_THRESHOLD	12	Intensity threshold (debug build: 0). Bypass when flops/(bytes*thr)<1. 0=always

`OZAKI_N`, `OZAKI_FLAGS`, `OZAKI_TRIM` and `OZAKI_MAXK` are passed to the GPU path, where they differ as follows (see the LIBXSTREAM Ozaki README): under `OZAKI=3`, `OZAKI_N` sets the moduli only and a positive `OZAKI_TRIM` applies to Scheme 1 only; auto `OZAKI_FLAGS` is 3 at every size (0 with `OZAKI=2`); and `OZAKI_MAXK` has no unbounded setting (0 selects the default) and also sizes the Scheme-2 moduli.

GPU Path (requires LIBXSTREAM)

Variable	Default	Description
OZAKI_OCL	0	Enable OpenCL/GPU path (0=off, 1=on)
OZAKI_TM	(auto)	GPU output tile height, only together with OZAKI_TN (not wgmma)
OZAKI_TN	(auto)	GPU output tile width, only together with OZAKI_TM (not wgmma)

GPU-specific kernel tuning variables (OZAKI_RTM, OZAKI_RTN, OZAKI_KU, OZAKI_RC, OZAKI_PB, OZAKI_HIER, OZAKI_GROUPS, OZAKI_PREFETCH, OZAKI_SCALAR_ACC, OZAKI_CACHE, OZAKI_ARENA, and more) are documented in the LIBXSTREAM Ozaki README. LIBXSTREAM reads them itself, except OZAKI_GROUPS, which this driver reads and passes to the GPU context; all of them take effect only with `OZAKI_OCL=1`.

Monitoring and Diagnostics

Variable	Default	Description
OZAKI_VERBOSE	0	0=silent, 1=stats at exit, N=print every Nth GEMM call
OZAKI_STAT	0	Track C-matrix (0), A-representation (1), or B-representation (2)
OZAKI_DUMP	0	Dump A/B as MHD files at the given call-count (0=off)
OZAKI_EPS	inf	Dump A/B when epsilon error exceeds threshold
OZAKI_IDX	0	Additional OZAKI_EPS metric: 1=linf_abs, 2=linf_rel, 3=l2_rel (0=epsilon only)
OZAKI_RSQ	0	Dump A/B when RSQ drops below threshold (updated after dump)
OZAKI_EXIT	1	Exit on accuracy violation after dump. 0=continue

With `OZAKI_OCL=1`, `OZAKI_VERBOSE` is also the verbosity of the GPU path: any non-zero value prints notes and warnings about its settings, and 3 or more (or a negative value) prints everything.

Benchmark

Variable	Default	Description
CHECK	0	Validate vs BLAS: 0=off, negative=auto-threshold, positive=custom
GRADE	0	Componentwise accuracy grade vs BLAS (see below)
XREF	0	Compensated reference on n rows, negative=all rows (see below)
NREPEAT	3	Number of GEMM calls (first call is warmup when >1)
EVIL	0	Adversarial exponent-span test (see below)
TAME	0	Keep n mantissa bits and clear the rest (see below)
SCALE	0	Multiply A and B by 2^n after generation
OZAKI_DECAY	0	Decay diagnostic + K-permutation (0-4, see below)
GEMM_HOSTMEM	0	Operands from LIBXSTREAM (page-locked) instead of `malloc`

By default, operands are allocated with `malloc`, and no device is opened unless the GPU path is enabled (`OZAKI_OCL=1`). A non-zero `GEMM_HOSTMEM` allocates page-locked memory through LIBXSTREAM, which opens a device.

Adversarial Input (EVIL) The EVIL variable initializes A and B with controlled exponent structure for stress-testing accuracy. The magnitude sets the exponent span in bits; the sign selects the distribution:

- `EVIL=N` (N>0): per-column. Column j of A is scaled by 2^(-N + round(2N*j/(ncols-1))), column j of B by the inverse, so the span is 2N binades centred on zero. The product AB is well-conditioned, and exponents are uniform within each column.
- `EVIL=-N` (N>0): per-element. Each element gets a pseudorandom exponent in [0,N] via coprime shuffle, with opposite sign for B. Every row of A spans the full exponent range.
- `EVIL=0`: default shuffle mode (no exponent structure).

The per-column mode (EVIL>0) is a diagonal scaling with D and D^-1, where EVIL is the exponent-range parameter.

`TAME=n` keeps n mantissa bits and clears the rest, e.g., `TAME=24` is data promoted from single precision. TAME and EVIL can be combined. Each run reports the generator version (`LIBXS_MATRNG_VERSION`) as `DATA: matrng=...`.

Accuracy Grade (GRADE) `GRADE=1` applies the componentwise criterion of the graded BLAS accuracy tests: `|C - C_ref| <= f(K) * u * (|alpha||A||B| + |beta||C|)` with `f(K) = K + 1024`, `u` the unit roundoff, and K the number of terms summed. It reports `GRADE: a=<max ratio> f(n)=<K>` and fails the run when the ratio exceeds it. `GRADE=-1` reports without failing (e.g., for runs with `OZAKI_TRIM`). By contrast, `CHECK` compares one scalar against a fixed threshold (1e-10 for double, 1e-3 for single). GRADE costs one extra reference GEMM and one m-by-n buffer.

Compensated Reference (XREF) `XREF=n` adds a reference computed by `libxs_dot2_gemm`, as if in twice the double precision, on n rows spread over C (`XREF=-1` takes all rows). It reports

XREF: rows=n ozaki_linf_rel=.. ozaki_l2_rel=.. ozaki_grade=.. blas_linf_rel=.. blas_l2_rel=.. blas_grade=..

for the Ozaki and the BLAS result. The grades appear with `GRADE` set, and `GRADE=1` then also fails the run on the compensated grade (`GRADE XREF:`). With beta non-zero, the reference replays the warmup and the timed calls. The cost is O(nNK) per replayed call on the host, hence a few rows suit large sizes.

Decay Diagnostic and K-Permutation (OZAKI_DECAY) Controls K-dimension row reordering for smoothness and the forward-difference decay diagnostic. Values map to libxs_sort_t:

Value	Behavior
0	Off (default). No permutation, no diagnostic.
1	Identity permutation. Measure decay on original ordering.
2	Sort B rows by L1-norm, apply same K-permutation to A.
3	Sort B rows by mean value, apply same K-permutation to A.
4	Greedy nearest-neighbor chain on B rows (O(K^2*N)).

Values 2-4 compute a K-permutation from B (via libxs_sort_smooth) before Ozaki-1 slicing; both A and B are sliced in the permuted K-order, and the product is unchanged.

When nonzero, reports the forward-difference decay of int8 slice buffers along K, M, and N axes (first K-group only, single- threaded). Output goes to stderr:

OZ1[MxNxK] Delta-K: d1=... d2=... ... OZ1[MxNxK] Delta-M: d1=... d2=... ... OZ1[MxNxK] Delta-N: d1=... d2=... ...

Profiling

Set `LIBXSTREAM_PROFILE=1` for per-kernel device timings, reported at program exit:

```
PROF ACC/OpenCL: ID=<id> <kernel>=<time> ms <rate> GFLOPS/s
```

There is no separate host-side profile: on the CPU, the `OZAKI GEMM` line reports the mean over the timed loop (see `NREPEAT`).

Example

```
./dgemm-wrap.x 256                                          # adaptive scheme (default)
OZAKI=1 ./dgemm-wrap.x 256                                  # mantissa slicing
OZAKI_TRIM=2 OZAKI=1 ./dgemm-wrap.x 256                     # trim 2 levels (less accurate)
OZAKI_OCL=1 OZAKI=2 OZAKI_GROUPS=4 ./dgemm-wrap.x 4096      # GPU CRT with K-grouping
OZAKI=3 ./dgemm-wrap.x 4096                                 # adaptive scheme selection
EVIL=512 ./dgemm-wrap.x 1024                                # accuracy grading (wide exponent span)
EVIL=1 ./dgemm-wrap.x 1024                                  # narrow exponent span
EVIL=-52 ./dgemm-wrap.x 1024                                # per-element exponent span
OZAKI_DECAY=1 OZAKI=1 ./dgemm-wrap.x 256                    # decay diagnostic (no reorder)
OZAKI_DECAY=2 OZAKI=1 ./dgemm-wrap.x 256                    # sort by norm + decay diagnostic
OZAKI_DECAY=4 OZAKI=1 ./dgemm-wrap.x 256                    # greedy NN + decay diagnostic
```

Pack

Measures bit packing (`libxs_pack.h`) at a fixed width against a width per group of values (`libxs_vpack`): bits per value, ratio, and time per value. Every round trip is verified.

Build

```
cd samples/pack
make
```

Usage

```
./pack.x [nvalues [nbits [range [zeros [nrepeat]]]]]
./pack.x FILE.mhd [FILE.mhd ...] [eps]
```

Without a file, the values are synthetic: signed integers of up to `nbits` bits whose magnitudes are log-uniform over `range` bits per block of 512 values, with `zeros` percent set to zero.

Positional	Default	Description
nvalues	1048576	Number of values
nbits	24	Width of the fixed format (2 to 63)
range	12	Spread of magnitudes in bits
zeros	10	Percentage of zeros
nrepeat	10	Repetitions per measurement

With MHD files (any element type), the values are measured in blocks of 512 and summed over all files. Integer data is packed as it is. Floating-point data is quantized to multiples of `eps` (values not above `eps` become zero), and each block gets the width its largest value needs. A file whose MHD extension holds a REAL(8) eps, an INT32 number of blocks, and an INT32 shape(3) per block provides eps and the blocks itself, so a dump of the data to analyze can carry its own blocking.

The output compares the fixed width with a width per group of 4 to 32 values, with deltas between neighbors, and for floating-point data with a quantization step of twice `eps`, plus the bits that a width per value would need. With MHD files, a second table splits each group size into the payload and the cost of storing the widths: one byte per width, 7 bits per width, a base per 8 groups with 3- or 2-bit offsets (`base+3`, `base+2`), and the entropy of the widths as a bound. Each row gives bits per value and the ratio.

Predict Samples

Twelve executables demonstrating fingerprint-guided prediction:

- **predict_params** — Parameter prediction from structured CSV (GPU kernel tuning, configuration databases).
- **predict_sunspots** — Timeseries forecasting via sliding-window kNN (monthly sunspot numbers, 1749–present).
- **predict_earthquakes** — Spatial prediction of earthquake magnitude from location and depth (USGS catalog).
- **predict_discharge** — River discharge forecasting via sliding-window kNN with day-of-year seasonality (USGS NWIS daily streamflow).
- **predict_soi** — Southern Oscillation Index prediction from anti-correlated Tahiti/Darwin sea level pressure using SPREAD decomposition (sum/diff modes).
- **predict_stock** — Paired-stock timeseries prediction from CSV using SPREAD decomposition on two correlated price series.
- **predict_crystal** — Crystal system classification from composition features (AFLOW ICSD, 7 classes, 60K entries).
- **predict_bits** — Held-out code length under the model against the training distribution: whether the inputs inform the output at all.
- **predict_ett** — ETT (Electricity Transformer Temperature) hourly forecasting with univariate, multivariate, PCA, and local-attention modes (ETTh1).
- **predict_prob** — Probability harness: scores P(y|x) for a candidate over the distinct-value support of one output (`libxs_predict_prob`).
- **predict_dist** — Validates `libxs_predict_prob`: normalization, agreement of its entry points, and sampling from the distribution.
- **predict_higgs** — Binary classification on the HIGGS dataset (28 features); `--help` prints the options.

Build

```
make
```

Or from the LIBXS root:

```
make samples/predict/
```

XGBoost Comparison

`predict_params`, `predict_crystal`, `predict_earthquakes`, `predict_ett` and `predict_higgs` accept an `xgb` keyword that trains XGBoost on exactly the entries LIBXS was built from and reports both models side by side. It is compiled in when XGBoost is found, which needs the development headers (a pip-installed `xgboost` ships the shared library only):

```
XGBOOST_ROOT=/path/to/xgboost make    # explicit location
make                                  # or found via pkg-config
XGB=0 make                            # never compile it in
```

Requesting `xgb` from a binary built without it is an error, not a silent LIBXS-only run.

Keywords may be written anywhere on the command line and numbers fill the positional slots in order, so `predict_crystal.x data.csv xgb` and `predict_crystal.x data.csv 0.8 2 0 xgb` are the same run. A token that is neither a number nor a known keyword is refused. `predict_params` is the exception in that such a token is taken as the next file name.

Per-output, XGBoost is posed the same task LIBXS chose for itself: a classification over the attested value set where LIBXS classifies, a regression where it interpolates. An output with a single attested value is reported as constant and not trained.

Hyperparameters are fixed so the numbers are reproducible, and overridable:

Variable	Default	Meaning
`XGB_ROUNDS`	200	Boosting rounds
`XGB_DEPTH`	6	`max_depth`
`XGB_ETA`	0.1	Learning rate
`XGB_NTHREAD`	0	Threads (0 = XGBoost default)
`XGB_REGOBJ`	*(per sample)*	Regression objective
`XGB_LATENCY`	4096	Rows the per-query timing uses (0 = skip)

Timings report XGBoost's phases apart (marshalling, boosting rounds, batched prediction); only the rounds compare to a LIBXS build. Per-query timings are reported both one query at a time (`libxs_predict_eval`, and `XGB_LATENCY` rows through XGBoost's inplace prediction) and batched (`libxs_predict_eval_batch_task`, and XGBoost's batched call); compare like with like.

`predict_earthquakes` proposes `reg:absoluteerror` for `XGB_REGOBJ`, since it reports mean absolute error; `predict_params` and `predict_crystal` propose `reg:squarederror`.

`predict_crystal`, `predict_higgs` and `predict_params` additionally read `GATE`: a comma-separated list of confidence thresholds (default `0.9`, and `0.5` for `predict_params`). The first entry drives the confidence-gated line; supplying more than one also prints a precision/coverage sweep for both models, which is what compares two confidence signals:

```
GATE=0.5,0.6,0.7,0.8,0.9,0.95,0.99 ./predict_crystal.x predict_crystal.csv 0.8 2 0 xgb
```

```
./predict_params.x 0.8 mix xgb /path/to/libxstream/samples/smm/params/tune_multiply_PVC.csv
./predict_crystal.x predict_crystal.csv 0.8 2 0 xgb
```

The parameter CSVs are not part of LIBXS: they ship with LIBXSTREAM in `samples/smm/params/`.

The parameter CSVs are sorted by problem size, so the default prefix split holds out the largest shapes and measures extrapolation. Use `mix` to measure interpolation instead.

`quantile` reports interval coverage split into attested and novel entries; only the novel column is a calibration.

`GFLOPS` is loaded as an output like every other column. A zero column means it is withheld and is hidden from the report. Where it is published, the report additionally weighs every shape by the throughput of its tuned parameters.

predict_params

Train a prediction model from a CSV file and save it for later use. Reports validation quality on a held-out subset.

```
./predict_params.x [fraction] [auto|cat|compress[Q]|consist[C]|interp|mix|quantile[Q]|rf|hknn|knn|smoo

fraction   Validation split 0..1 for quality report (default: 0.8).
auto       Auto-detect mode per output (default).
cat        Force categorical (kNN) for all outputs.
compress   Drop redundant entries (Q: threshold, default 0.9).
consist    Round-trip consistency penalty (C: 0..1, default 0.9).
interp     Force interpolation for all outputs.
mix        Draw the validation split by co-prime stride rather than as a prefix.
quantile   Prediction intervals (Q: level 0..0.5, default 0.1).
rf         Random Forest classification.
hknn       Hierarchical kNN (Fisher-guided partition).
knn        Plain kNN.
smooth     Multi-cluster blending (A: radius, or -1 = auto; default -1).
xgb        Also train XGBoost on the same split and compare.
-N         Max polynomial order (default: 0 = auto).
csvfile    Delimited text file.
modelfile  Output path for the binary model.
confidence-prefix  Optional prefix for confidence map files: MHD, and PNG for the slices.
```

Without `rf`, `hknn` or `knn`, the build chooses the method by trial on entries it holds back, as the other samples do. Naming a mode fixes it: the saved model keeps only what that mode reads and cannot be evaluated by another one. A chosen mode leaves the model open to all of them.

```
./predict_params.x /path/to/libxstream/samples/smm/params/tune_multiply_PVC.csv
./predict_params.x 0.8 hknn tune_multiply_PVC.csv model.bin
```

predict_sunspots

Timeseries forecasting using sliding-window nearest-neighbor prediction.

```
./predict_sunspots.x <csvfile> [train_fraction] [compress[Q]] [consist[C]] [hknn|rf|none]

NOPHASE=1  Drop the solar-cycle phase input.
NOBANK=1   Use a single window view instead of the bank.
WINDOW=n   Fix the window (default: auto). WINDOW=-1 selects it by
           trial instead, at a multiple of the build time. WINDOW=-w
           also offers w as a candidate.
```

```
./predict_sunspots.x predict_sunspots.csv 0.8
```

The sample carries one auxiliary input alongside the window (libxs_predict_set_series_aux): the phase of the solar cycle, measured from the training slice. NOPHASE=1 drops it.

The forecast is averaged over two views of the window (libxs_predict_set_series_bank), the second reading half the lags of the first. NOBANK=1 drops back to one. Views are unavailable under a decomposition.

Data Source Monthly mean total sunspot number from SILSO (World Data Center, Royal Observatory of Belgium). Semicolon-delimited: year, month, decimal_year, sunspot_number.

predict_earthquakes

Predict earthquake magnitude from geographic location and depth.

```
./predict_earthquakes.x <usgs_csv> [train_fraction] [compress[Q]] [consist[C]] [hknn|rf|none] [xgb]

consist    Round-trip consistency penalty (C: 0..1, default 0.9).
none       Plain kNN (otherwise the mode is selected at build by trial).
xgb        Also train XGBoost on the same split and compare.
```

The kNN vote reports median or mean as libxs_predict_set_central picks per output at build time (passing 1 or 2 to it forces median or mean).

```
./predict_earthquakes.x predict_earthquakes.csv
```

Data Source USGS Earthquake Hazards Program (public domain). Comma-delimited: time, latitude, longitude, depth, mag, ...

predict_discharge

River discharge forecasting with day-of-year seasonality and log-transform on outputs for heavy-tailed data.

```
./predict_discharge.x <discharge_tsv> [train_fraction] [compress[Q]] [consist[C]] [hknn|rf|none]

WINDOW=n   Fix the window (default: auto). A negative value selects it by
           trial; WINDOW=-w also offers w as a candidate.
BANK=n     Average the forecast over n views of the window (default: 1).
```

```
./predict_discharge.x predict_discharge.tsv
```

Data Source USGS National Water Information System (public domain). Colorado River at Lees Ferry, site 09380000. Tab-delimited RDB format.

predict_soi

Southern Oscillation Index prediction from anti-correlated sea level pressure at Tahiti and Darwin using SPREAD decomposition.

```
./predict_soi.x <tahiti_file> <darwin_file> [train_fraction] [compress[Q]] [consist[C]] [hknn|rf]

WINDOW=n   Fix the window (default: auto). A negative value selects it by
           trial; WINDOW=-w also offers w as a candidate.
BANK=n     Average the forecast over n views of the window (default: 1).
```

```
./predict_soi.x predict_soi_tahiti.dat predict_soi_darwin.dat
```

Data Source NOAA Climate Prediction Center (public domain). Fixed-width monthly sea level pressure (mb above 1000 mb).

predict_stock

Multi-stock timeseries prediction with auto-differencing and PCA/SPREAD decomposition.

```
./predict_stock.x <csv_file> [columns] [train_fraction] [compress[Q]] [consist[C]] [hknn|rf]

columns    Comma-separated 0-based column indices (default: 1,2).
```

```
./predict_stock.x stocks.csv 1,2,3
```

predict_crystal

Crystal system prediction (7-class classification) from chemical composition features.

```
./predict_crystal.x <crystal_csv> [train_fraction] [order] [nclusters] [compress[Q]] [consist[C]] [fis

consist     Round-trip consistency penalty (C: 0..1, default 0.9).
fisher      Fisher discriminant feature weighting.
hknn        Hierarchical kNN (Gini-guided partition).
setdiff     Setdiff feature selection.
rf          Random Forest classification.
pca         Principal component rotation of the inputs.
none        Raw kNN without feature processing (otherwise the mode is
            selected at build by trial).
rank        Neighbour modes place inputs by rank.
score       Neighbour modes place inputs by what they say about the class
            (default: chosen by trial between the two from 8 inputs on).
gaps        Blank that fraction of input values (F: default 0.1) to exercise
            the absent-value path (libxs_predict_set_missing).
xgb         Also train XGBoost on the same split and compare.
```

```
./predict_crystal.x predict_crystal.csv
```

Data Source AFLOW ICSD catalog (free for academic use). 60,386 entries with Magpie-style composition features (37 features). Crystal systems: triclinic(1), monoclinic(2), orthorhombic(3), tetragonal(4), trigonal(5), hexagonal(6), cubic(7).

predict_bits

Code length of a held-out stream in bits per event, under the model and under the training distribution of the same output. Use it to ask whether the inputs inform the output at all, which an error figure does not answer.

```
./predict_bits.x <csvfile> <innames> <outname> [fraction] [vocabulary] [hknn]

vocabulary  Distinct values the caller considers possible. 0 uses the
            attested support plus one aggregate novel atom, so an
            unattested outcome costs infinite bits; pass a count above
            the support size to keep the figure finite.
hknn        Non-zero selects the hierarchical partition.
```

Example Run a control alongside any such measurement: a null result is also what a broken setup reports, so pair the question with one whose answer is known.

```
./predict_bits.x predict_earthquakes.csv latitude,longitude,depth mag 0.8 68
./predict_bits.x predict_earthquakes.csv latitude,longitude,mag depth 0.8 9000
```

predict_ett

ETT (Electricity Transformer Temperature) hourly forecasting. Supports univariate and multivariate modes with PCA decomposition and per-query local-correlation attention.

```
./predict_ett.x <ett_csv> [nseries=1..7] [attend|spread|pca|hknn|rf|none|compress[Q]|xgb]

nseries     Number of input channels (1=OT only, 7=all).
attend      Per-query local-correlation channel weighting.
spread      Sum/diff decomposition across channels.
pca         PCA rotation of multi-channel input space.
none        No decomposition (otherwise the mode is selected at build by
            trial, which costs one full build per candidate per fold).
compress    Drop redundant entries (Q: threshold, default 0.9); off unless asked.
xgb         Also train XGBoost on the same windows and compare.
WINDOW=n    Fix the window (default: auto). A negative value selects it by
            trial; WINDOW=-w also offers w as a candidate.
BANK=n      Average the forecast over n views of the window (default: 1).
```

`xgb` overrides `compress` and is refused alongside `attend`, a decomposition, or `BANK`. The effective window applies to both models; `WINDOW=96` compares them at the conventional input length rather than at the auto-selected one.

```
./predict_ett.x predict_ett.csv              # univariate OT
./predict_ett.x predict_ett.csv 2 pca        # 2ch with PCA
./predict_ett.x predict_ett.csv 7 attend     # 7ch with local-attention
./predict_ett.x predict_ett.csv 7            # 7ch raw (baseline)
```

Data Source ETTh1 (Electricity Transformer Temperature, hourly) from Zhou et al. 2021 (Informer). 17,420 hourly readings of an oil-filled electrical transformer (July 2016 – June 2018). Seven channels: HUFL, HULL, MUFL, MULL, LUFL, LULL, OT. Target: OT (oil temperature). The sample trains on the first 66.1% (months 1–16, the standard training and validation months) and tests on the rest, horizon H=96 steps (4 days).

`predict_ett.csv` is ETTh1 as published at github.com/zhouhaoyi/ETDataset/blob/main/ETT-small/ETTh1.csv (a fresh download with CRLF line endings is accepted as well).

Registry Microbenchmark

Benchmarks the performance of the LIBXS registry (key-value store) dispatch path under different access patterns and concurrency levels.

Building

```
cd samples/registry
make
```

Produces `registry.x` (C). The Fortran variant `registryf.x` is built only when a Fortran compiler is found and the library was built beforehand: without a built library the sample is compiled header-only and the Fortran variant is skipped.

Programs

```
./registry.x [total [nrepeat [nthreads [keysize]]]]
```

Argument	Default	Description
total	10000	Number of unique keys to register (min 2)
nrepeat	10	Repeat iterations for lookup phases
nthreads	max available	Number of OpenMP threads
keysize	sweep	Pin one key size in Bytes (8 to 64)

Without `keysize`, the single-threaded phases sweep key sizes of 8, 12, 16, 28, 32, 48 and 64 Bytes and print one row per size. A pinned `keysize` prints the per-phase detail and the registry's size, capacity and footprint instead. The multi-threaded phases use the pinned size, or 12-Byte keys by default.

Measurements:

- Duration to register (insert) all keys into the registry.
- Cold lookup with shuffled access pattern.
- Locked lookup with sequential/repeating pattern (small working set).
- Multi-threaded parallel reads across all threads.
- Contended parallel writes (each thread registers its own key range).
- Mixed read/write: one writer thread while remaining threads read.

The multi-threaded benchmarks require at least 2 threads and are skipped when running single-threaded.

```
./registryf.x
```

Fortran variant with hardcoded parameters (10000 keys, 10 repeats). Measures registration, cold lookup, and locked lookup.

Rosetta

Demonstrates hierarchical type discovery on opaque binary data. A table of harmonic-series values is encoded as a flat byte blob with no metadata. The analysis rediscovers the element type, record stride, and per-field structure — recovering mathematical content from anonymous bytes.

What It Shows

The sample proceeds through these levels:

Step	Tool used	What it checks
Flat probe	libxs_fprint (UNKNOWN)	smoothness of the bytes as one array
Stride sweep	libxs_fprint per-column	element type and record stride
Shuffle test	libxs_shuffle + fprint	whether the record order matters
Field analysis	libxs_fprint per-field	decay rate of each column
Sort test	libxs_sort_smooth (GREEDY)	whether a reordering is smoother
Verification	libxs_setdiff_min	recovered records against the blob

The Data

For n = 1, 2, ..., 64 the table stores six f64 fields per record:

```
[0] n             integer index
[1] 1/n           harmonic term
[2] H_n           partial sum of the harmonic series
[3] ln(n)         natural logarithm
[4] H_n - ln(n)   difference (converges to gamma from above)
[5] H_n - ln(n) - 1/(2n)   better gamma estimate
```

Total: 64 records x 6 fields = 384 doubles = 3072 bytes.

Build

```
cd samples/rosetta
make
```

Usage

```
./rosetta.x
```

No arguments. The sample is self-contained: it generates the data, encodes it as a flat blob, runs the full analysis, and prints the results.

To collect machine-readable artifacts, set `ROSETTA_OUTDIR` to an existing directory:

```
mkdir -p results/rosetta
ROSETTA_OUTDIR=results/rosetta ./rosetta.x
```

This writes `summary.csv`, the record matrices `rosetta_original.mhd` and `rosetta_shuffled.mhd`, and the images `rosetta_fields.png` and `rosetta_fields_shuffled.png`. The `fields` images are normalized per field, making the ordered and shuffled record traces easy to compare visually. The artifact mode is optional and leaves the interactive sample output unchanged.

Memory Allocation (Microbenchmark)

Benchmarks pooled scratch memory allocation (`libxs_malloc` / `libxs_free`) against the standard C library `malloc` / `free`. The pool-based allocator recycles memory after an initial warm-up, avoiding repeated system calls in steady state. A Fortran variant compares `libxs_malloc` against Fortran ALLOCATE / DEALLOCATE.

Building

```
cd samples/scratch
make
```

Produces `scratch.x` (C). The Fortran variant `scratchf.x` is built only when a Fortran compiler is found (GNU Fortran needs 4.6 or later) and the library was built beforehand: without a built library the sample is compiled header-only and the Fortran variant is skipped.

Usage (C)

```
./scratch.x [ncycles [max_nactive [nthreads]]]
```

Argument	Default	Description
ncycles	100	Number of allocation/deallocation cycles
max_nactive	4	Max concurrent live allocations per cycle (max 24)
nthreads	1	OpenMP threads (clamped to omp_get_max_threads)

Environment Variables

Variable	Default	Description
CHECK	0	Non-zero: memset each allocation (validates writeable)

Compile-Time Knobs

Macro	Default	Description
MAX_MALLOC_MB	100	Upper bound on individual allocation size in MB
MAX_MALLOC_N	24	Array size for random-number table and alloc slots
___TBB	undefined	Baseline is TBB `scalable_malloc` instead of malloc

```
## default: 100 cycles, up to 4 active allocations, 1 thread
./scratch.x

## heavy load: 500 cycles, up to 12 active allocations, 4 threads
./scratch.x 500 12 4

## validate pages are writable
CHECK=1 ./scratch.x 43 8 4
```

Usage (Fortran)

```
./scratchf.x [mbytes [nrepeat]]
```

Argument	Default	Description
mbytes	100	Allocation size in MB
nrepeat	20	Number of repetitions

Single-threaded benchmark comparing `libxs_malloc` / `libxs_free` against Fortran ALLOCATE / DEALLOCATE. Reports per-iteration times (ms) and an average speedup ratio.

What Is Measured

Each cycle draws a random number of allocations (1 to max_nactive) with random sizes (1 to MAX_MALLOC_MB MB each). The cycle allocates all buffers, optionally touches them (CHECK), then frees them.

Both paths use the same randomized sequence. An untimed warm-up cycle runs before the measured loops. Reported metrics:

- calls/s (kHz) — allocation+free throughput for each allocator
- Scratch size — high-water mark of the pool (MB)
- Malloc size — peak aggregate allocation per cycle (MB)
- Scratch Speedup — ratio of pool throughput to stdlib throughput
- Fair — size-adjusted speedup: (malloc_size / scratch_size) * speedup

Setdiff

Runs small deterministic experiments for `libxs_setdiff` and `libxs_setdiff_min` and writes the results as CSV files.

Build

```
cd samples/setdiff
make
```

Usage

```
mkdir -p results/setdiff
./setdiff.x [outdir [sizes [reps [land_n [land_steps]]]]]
```

Positional	Default	Description
outdir	`results/setdiff`	Directory for CSV output
sizes	`128 1024 8192 65536`	Vector sizes for tolerance and scaling runs, as one quoted argument separated by spaces
reps	`10`	Repetitions per scaling size
land_n	`32`	Tolerance-landscape vector length
land_steps	`40`	Number of tolerance grid steps

Output

- `summary.csv` — order-independence and duplicate-consumption cases
- `landscape.csv` — sampled tolerance landscape
- `tolerance.csv` — `libxs_setdiff_min` compared with bisection
- `scaling.csv` — fixed-tolerance and automatic-tolerance timings
- `complex.csv` — complex-valued validation case

The output directory must exist; the sample does not create it and fails if it cannot write there. The sample uses a fixed pseudo-random seed, and timings in `scaling.csv` are CPU time (`clock`).

Shuffle

Benchmarks three shuffling strategies and compares their throughput:

Label	Method
RNG-shuffle	Fisher-Yates via `libxs_rng_u32` (reference baseline)
DS1-shuffle	`libxs_shuffle` — deterministic in-place (coprime stride)
DS2-shuffle	`libxs_shuffle2` — deterministic out-of-place (src to dst)

Each iteration works on an array of unsigned integers initialized to 0..n-1. On the final iteration the shuffled result can be written as a PNG image (see below). Every run ends with round-trip checks of `libxs_shuffle2`/`libxs_unshuffle2`, once with the given offset and once with offset 42, each reported as PASS or FAIL.

Build

```
cd samples/shuffle
make
```

Usage

```
./shuffle.x [nelems [elemsize [niters [repeat]]]]
```

Positional	Default	Description
nelems	64 MB / elemsize	Number of elements
elemsize	4	Element size in bytes (1, 2, 4, or 8)
niters	1	Shuffle passes per timed measurement
repeat	3	Timed iterations after one warm-up

Environment Variables

Variable	Default	Description
BIAS	0	Coprime selection in [-1, +1] (`libxs_coprime_bias`)
OFFSET	0	Offset of the deterministic shuffles
RANDOM	0	Non-zero: count inversions and report randomness (rand=%)
SPLIT	1	Partitioning depth for the STATS metrics
STATS	0	Non-zero: print partition imbalance (imb) and distance (dst)

BIAS=-1 selects the smallest non-trivial coprime, 0 one near sqrt(n), and +1 one near n/2. If BIAS, OFFSET, RANDOM or STATS is non-zero, the sample first prints n, the selected coprime, bias and offset.

```
## Default run (64 MB of 4-byte elements, 3 repeats)
./shuffle.x

## 1M elements of 8 bytes, 4 shuffle passes, 5 repeats
./shuffle.x 1000000 8 4 5

## Enable randomness quality metric
RANDOM=1 ./shuffle.x 10000 4 1 3

## Enable partition statistics with split depth 2
STATS=1 SPLIT=2 ./shuffle.x
```

What Is Measured

Reported bandwidth is 2 * data-size / time (in MB/s, with 1 MB = 2^20 Bytes). The factor of two accounts for reading and writing each element during the shuffle. The sample runs `repeat + 1` iterations: the first is a warm-up, and when `repeat` is larger than 1 the average over the remaining `repeat` iterations is printed at the end.

Optional quality metrics:

- rand — Enabled by RANDOM=1. Inversions are counted via merge-sort and compared against the expected count for a random permutation (n*(n-1)/4) to give a percentage.
- dst — Manhattan distance of the element sum from the expected uniform value, split into hierarchical partitions (SPLIT).
- imb — Partition imbalance, measuring how unevenly element sums are distributed across sub-partitions (SPLIT).
- With STATS=1, the sample also prints the fingerprint distance (Sobolev, order 4) between the original and each shuffled array, and between the RNG-shuffled and each deterministic result.

Image Output When RANDOM is not set and the element size is 1, 2, 4, or 8 bytes, the final shuffled array is written as a PNG image per method (shuffle_rng.png, shuffle_ds1.png, shuffle_ds2.png). The images are shaped close to square (isqrt(n) x n/isqrt(n)); each pixel is an 8-bit hash of the element value and its position.

Compile-Time Knobs

Variable	Default	Description
OMP	0	Enable OpenMP (not used by this sample)
SYM	1	Include debug symbols (-g)
BUBBLE_SORT	undefined	Define (-DBUBBLE_SORT) for O(n^2) inversion counter

Spatial

Samples for spatial data structures and queries built on libxs primitives: space-filling curves (Hilbert, Morton), kd-tree partition and search, and spatial pair counting. Each topic uses a filename prefix to group related files in this flat directory.

Prefix	Topic
`stratify_*`	3D-to-2D layout via space-filling curves
`paircnt_*`	Spatial pair counting

Underlying libxs primitives (from `libxs_perm.h`):

- Hilbert and Morton curve encode/decode (N-D stratification)
- kd-tree build, partition, nearest-neighbor search
- Sort by spatial locality (various methods)

Building

The default target builds the C sample `paircnt_dd.x` (header-only when the library is not built). The Python scripts load the shared library, so build libxs with `STATIC=0` first so that `lib/libxs.so` exists:

```
cd ../..
make PEDANTIC=2 STATIC=0
cd samples/spatial
make
```

The scripts look for the library given by `--libxs PATH`, then by the environment variable `LIBXS_LIBRARY`, then in `lib/` of the repository, then on the system search path.

Optional Python adapters need `numpy` and `h5py`; `paircnt_validate.py` also needs `corrfunc` (which needs `libgsl-dev` to build). All three are listed in `requirements.txt`:

```
python3 -m venv .venv
. .venv/bin/activate
python -m pip install -r requirements.txt
```

Stratification (stratify_*)

Folding higher-dimensional data into lower-dimensional layouts using space-filling-curve stratification. The mapping can be computed once for a fixed detector or simulation grid and reused by any downstream pipeline.

The transform encodes each source voxel coordinate as a 3D Hilbert or Morton key and decodes it as a 2D coordinate:

```
3D coordinate -> 3D curve rank -> inverse 2D curve coordinate
```

The source is a regular `D,H,W` index grid. The default `compact` frame fills a dense near-square sheet in curve-rank order; the `canonical` frame places each voxel at the 2D curve coordinate of its rank, which can leave cells empty.

Usage The `dense3d` target runs the synthetic dense 3D sample:

```
make dense3d
make dense3d FRAME=canonical
```

The older `make volume` spelling remains as a compatibility alias.

Direct invocation:

```
python3 stratify_dense3d.py --shape 8 16 16 --curve hilbert \
  --frame compact --out sheet.pgm
python3 stratify_dense3d.py --shape 8 16 16 --curve morton \
  --frame canonical --map-csv map.csv
```

HDF5 input is optional and requires `h5py` and `numpy`. The input dataset may be a single `D,H,W` volume, a batched `N,D,H,W` dataset, a channelled `N,C,D,H,W` or `N,D,H,W,C` dataset, or a flattened `N,V` dataset when `--hdf5-reshape D H W` is supplied. The default `auto` layout recognizes the 3DGAN Caffe sample layout `N,C,D,H,W`:

```
python3 stratify_dense3d.py --hdf5 /path/to/train.h5 \
  --hdf5-dataset ECAL --hdf5-event 0 --hdf5-channel 0 \
  --curve hilbert --frame compact --out sheet.pgm --out-hdf5 sheet.h5
```

The Makefile exposes the same path without making the default target depend on external data:

```
make hdf5 HDF5_FILE=/path/to/train.h5 FRAME=compact
```

For flattened HDF5 files, pass the dataset name, layout, and target 3D shape:

```
make hdf5 HDF5_FILE=dataset_2_1.hdf5 HDF5_DATASET=showers \
  HDF5_LAYOUT=flat HDF5_RESHAPE="45 16 9"
```

Public calorimeter HDF5 datasets with a documented download path are available from the CaloChallenge project. The challenge HDF5 files contain `incident_energies` and flattened `showers` datasets, so use `--hdf5-dataset showers` and `--hdf5-reshape D H W` with the geometry stated for the selected dataset. Useful starting points are:

Source	Format / notes
CaloChallenge	Overview, geometry, evaluation.
Dataset 1	ATLAS photons/pions, flat showers.
Dataset 2	Electrons, reshape to `45 16 9`.
Dataset 3	Hi-gran electrons, `45 50 18`.
Legacy sets	Submitted outputs.

Arguments:

Option	Description
`--shape D H W`	Volume shape (default `8 16 16`).
`--curve`	`hilbert` (default) or `morton`.
`--frame`	`compact` (default) or `canonical`.
`--libxs`	Path to libxs shared library.
`--hdf5`	Read volume from an HDF5 file.
`--hdf5-dataset`	HDF5 dataset name (default `ECAL`).
`--hdf5-layout`	`auto`/`dhw`/`ndhw`/`ncdhw`/`ndhwc`/`flat`.
`--hdf5-reshape`	Reshape flat data to `D H W`.
`--hdf5-event`	Event index (default `0`).
`--hdf5-channel`	Channel index (default `0`).
`--out`	Write 8-bit PGM of the sheet.
`--map-csv`	Write `src,z,y,x,dst,v,u` rows.
`--out-hdf5`	Write sheet+map to HDF5 file.

The Makefile targets take these variables:

Variable	Default	Used by
`PYTHON`	`python3`	all script targets
`FRAME`	`compact`	all script targets
`HDF5_FILE`	(required)	`hdf5`
`HDF5_DATASET`	`ECAL`	`hdf5`
`HDF5_LAYOUT`	`auto`	`hdf5`
`HDF5_RESHAPE`	(none)	`hdf5`
`HDF5_EVENT`	`0`	`hdf5`
`HDF5_CHANNEL`	`0`	`hdf5`
`MEDMNIST3D_NPZ`	(none)	`medmnist3d`, `medmnist3d-metrics`
`MEDMNIST3D_ROOT`	`~/.medmnist`	`medmnist3d`, `medmnist3d-metrics`

Variable	Default	Used by
MEDMNIST3D_FLAG	organmnist3d	medmnist3d, medmnist3d-metrics
MEDMNIST3D_SIZE	28	medmnist3d, medmnist3d-metrics
MEDMNIST3D_SPLIT	train	medmnist3d, medmnist3d-metrics
MEDMNIST3D_INDEX	0	medmnist3d, medmnist3d-metrics

MedMNIST3D The stratify_medmnist3d.py sample reads one standardized MedMNIST3D volume from an .npz file and applies the same 3D-to-2D stratification primitive. Only numpy is required; PyTorch or the medmnist package are needed only if you use them to download the data.

The official MedMNIST distribution is available from the project page and Zenodo: MedMNIST and Zenodo DOI. The 3D subsets are organmnist3d, nodulemnist3d, adrenalmnist3d, fracturemnist3d, vesselmnist3d, and synapsemnist3d.

Direct invocation with an explicit NPZ file:

```
python3 stratify_medmnist3d.py --npz ~/.medmnist/organmnist3d.npz \
  --split train --index 0 --curve hilbert --frame compact \
  --out stratified_medmnist3d.pgm \
  --map-csv stratified_medmnist3d.csv \
  --label-csv stratified_medmnist3d_label.csv
```

The Makefile exposes the same path without making the default target depend on external data:

```
make medmnist3d MEDMNIST3D_NPZ=~/.medmnist/organmnist3d.npz FRAME=compact
```

The same NPZ input can be used with the metrics script to report invariants, locality distortion, and LIBXS fingerprint distances for a selected volume:

```
make medmnist3d-metrics MEDMNIST3D_NPZ=~/.medmnist/organmnist3d.npz \
  MEDMNIST3D_SPLIT=test MEDMNIST3D_INDEX=0 FRAME=canonical
```

If MEDMNIST3D_NPZ is omitted, the script looks for a dataset under MEDMNIST3D_ROOT using MEDMNIST3D_FLAG and MEDMNIST3D_SIZE:

```
make medmnist3d MEDMNIST3D_ROOT=~/.medmnist MEDMNIST3D_FLAG=nodulemnist3d
```

Arguments of stratify_medmnist3d.py:

Option	Description
--npz	MedMNIST3D NPZ file (otherwise --root/--flag).
--root	Directory of NPZ files (default ~/.medmnist).
--flag	3D subset (default organmnist3d).
--size	Image size used with --root (default 28).
--split	train, val, or test (default train).
--index	Sample index within the split (default 0).
--curve	hilbert or morton.
--frame	compact or canonical.
--libxs	Path to libxs shared library.
--out	Write 8-bit PGM of the sheet.
--map-csv	Write the source-to-destination map as CSV.
--label-csv	Write the selected label as CSV.
--out-hdf5	Write sheet+map to HDF5 file.

Metrics The stratify_dense3d_metrics.py script reports invariants and locality distortion for the same synthetic and HDF5 inputs:

```
make metrics
python3 stratify_dense3d_metrics.py --hdf5 /path/to/train.h5 \
  --hdf5-dataset ECAL --curve hilbert
python3 stratify_dense3d_metrics.py --hdf5 dataset_2_1.hdf5 --hdf5-dataset showers \
  --hdf5-layout flat --hdf5-reshape 45 16 9 --curve morton
```

The invariant metrics check that stratification is a lossless layout transform: the total energy, reconstructed voxel values, and per-axis energy profiles match after applying the inverse map. The distortion metrics measure what changes for a convolutional model: how far source 3D grid neighbors move apart in the 2D sheet, and how often adjacent sheet cells correspond to adjacent source voxels.

The script also reports LIBXS fingerprint metrics. These are compact multi-order L2 descriptors of value and finite-difference structure. The `fprint.source_reconstructed.diff` value should be zero for a correct lossless round trip, while `fprint.source_sheet.diff` summarizes how different the stratified 2D sheet looks as a structured field.

The metrics script takes `--shape`, `--curve`, `--frame`, `--libxs`, and the `--hdf5*` options of `stratify_dense3d.py`. A MedMNIST3D volume is selected with `--medmnist3d-npz FILE`, or with `--medmnist3d` and `--medmnist3d-root`, `--medmnist3d-flag`, `--medmnist3d-size`; `--medmnist3d-split` and `--medmnist3d-index` pick the sample. `--csv FILE` writes the metrics as CSV.

Pair Counting (paircnt_*)

Counts the pairs of a point catalog per separation bin (DD), using the libxs kd-tree and skipping cell pairs that cannot fall into any bin.

```
python3 paircnt_gendata.py --npoints 10000 --boxsize 420.0 --out catalog.dat
./paircnt_dd.x catalog.dat --rmin 0.1 --rmax 25 --nbins 20
python3 paircnt_validate.py catalog.dat
```

`paircnt_dd.x <catalog> [--rmin R] [--rmax R] [--nbins N] [--leaf N]` reads a catalog of `x y z` lines, or a Fortran fastfood file when the name ends in `.ff`, and prints one `rmin rmax npairs` row per logarithmic bin. Timings and the total pair count go to stderr.

Option	Default	Description
`--rmin`	`0.1`	Smallest bin edge (greater than zero).
`--rmax`	`25.0`	Largest bin edge (greater than `--rmin`).
`--nbins`	`20`	Number of bins (at most 128).
`--boxsize`	`0`	Periodic box: not supported (rejected).
`--leaf`	`64`	Smallest kd-tree leaf (1 to 256).

`paircnt_gendata.py` writes uniform points (`--clustered` with `--nclusters` for clustered ones) with `--npoints`, `--boxsize`, `--seed` (default `42`), and `--out` (`.ff` selects the fastfood format). `paircnt_validate.py` runs `paircnt_dd.x` (`--exe`, default `./paircnt_dd.x`) and Corrfunc's non-periodic `DD` with the same `--rmin`, `--rmax`, and `--nbins` on a text catalog, compares the counts bin by bin, and exits non-zero on a mismatch.

Synchronization Primitives

Two binaries over `libxs_sync.h`: `sync_lock.x` for the lock kinds and `sync_barrier.x` for `libxs_barrier_t`.

sync_lock.x

Micro-benchmark for the lock implementations provided by LIBXS (`libxs_sync.h`). Measures single-thread latency (uncontended acquire/release) and multi-thread throughput (mixed read/write workload) for each lock kind:

Lock kind	Description
LIBXS_LOCK_DEFAULT	Compile-time default (typically atomic)
LIBXS_LOCK_SPINLOCK	OS-native or CAS-based spin lock (if available)
LIBXS_LOCK_MUTEX	OS-native mutex / pthread_mutex_t (if available)
LIBXS_LOCK_RWLOCK	Reader/writer lock / pthread_rwlock_t

All four kinds are always benchmarked. Platform support only decides whether a kind is the OS-native primitive or LIBXS's own implementation, which the output labels "OS-native" or "Other".

Build

```
cd samples/sync
make
```

OpenMP is enabled by default (OMP=1) for multi-threaded tests.

Run

```
./sync_lock.x [nthreads] [wratio%] [work_r] [work_w] [nlat] [ntpt]
```

Argument	Default	Description
nthreads	all available	Number of OpenMP threads
wratio%	5	Percentage of write operations (0-100)
work_r	100	Simulated work inside read-critical section (cy)
work_w	10 * work_r	Simulated work inside write-critical section
nlat	2000000	Iterations for latency measurement
ntpt	10000	Iterations per thread for throughput measurement

```
./sync_lock.x 4 5 100 1000
```

Each measured line names the lock kind it belongs to. The run first prints the pause budgets it was built with (lock and barrier), which a contended figure depends on; compare runs only when they match.

For a short, write-only critical section, as taken by a hot internal lock:

```
./sync_lock.x 16 100 1 1 100000 5000
```

Measurement Details

- RO-latency: uncontended read-lock acquire/release pairs (4x unrolled), reported as nanoseconds per operation and TSC cycles.
- RW-latency: uncontended write-lock acquire/release pairs (4x unrolled).
- Throughput: all threads run a mixed read/write workload governed by wratio%. Simulated work inside the critical section is subtracted so only synchronization overhead is reported.

sync_barrier.x

Cost of a rendezvous over `libxs_barrier_t`, and of the broadcast that hands one task's value to the rest.

```
./sync_barrier.x [nthreads] [nrepeat]
```

Argument	Default	Description
nthreads	all available	Tasks in the team
nrepeat	100000	Rendezvous per measurement

The output gives the cost per rendezvous in nanoseconds and cycles, once for `wait:` (plain rendezvous) and once for `bcast:` (rendezvous with a broadcast value).

The team size is taken from the runtime, which may grant fewer threads than requested. The stamp each task writes per rendezvous is checked in an untimed pass and every broadcast value is checked; the binary fails, reporting stale stamps or wrong broadcasts, if one is wrong. Expect the barrier to cost more once the team exceeds the number of cores.

SYRK / SYR2K Sample

Demonstrates symmetric rank-k and rank-2k updates using the LIBXS dispatch-and-call model. The sample validates correctness against a plain Fortran reference and reports performance.

Operations

SYRK: C := alpha * A * A^{T} + beta * C (lower triangle) SYR2K: C := alpha * (A * B^{T} + B * A^T) + beta * C (upper triangle)

Only the specified triangle of C is written; the other triangle is left untouched.

Build

```
make
```

Requires a Fortran compiler (gfortran, ifort, or ifx) and LIBXS built first from the repository root (the sample is not built header-only). The Makefile picks up the compiler from the top-level Makefile.inc. BLAS is linked dynamically and sequential by default (`BLAS=1`, `BLAS_STATIC=0`). JIT-compiled kernels need `make MKL=1` (sequential) or `make MKL=2` (threaded) for MKL's JIT, or `make LIBXSMM=1` with a built sibling `libxsmm` directory.

Run

```
./syrk.x [N [K [nrepeat [direct [tasks]]]]]
```

Arguments (all optional, positional):

```
N         Matrix dimension of C (N x N).  Default: 64
K         Inner dimension (columns of A).  Default: N
nrepeat   Number of timed repetitions.     Default: 100
direct    Interface selection.             Default: 0
            0 = config (dispatch + call separately)
            1 = direct (one-shot generic)
tasks     Tasks per thread (OpenMP).       Default: 4
```

OpenMP

When the Fortran module is compiled with OpenMP (the sample's default), `libxs_syrk` and `libxs_syr2k` run over an OpenMP team, unless called from within a parallel region. The team size comes from `libxs_syrk_ntasks`, which returns 1 (no split) for small shapes, with a threaded BLAS, or with too few threads against a sequential BLAS. The sample prints this count as `libxs_syrk=`.

The performance section compares `DSYRK` with `LIBXS` (`libxs_syrk`). `DSYRK` is threaded only with a threaded BLAS, e.g., `make MKL=2`:

```
make MKL=2
OMP_NUM_THREADS=16 OMP_PROC_BIND=close ./syrk.x 2048 2048 5
```

The `OMP` and `TASK` lines call `libxs_syrk_task` explicitly, as C code would: one task per thread, or `tasks` per thread over a dynamic DO.

Example Output

```
SYRK: N=64 K=64 nrepeat=100 direct=0

--- libxs_syrk (lower) ---
  max error (lower): 0.00000E+00

--- libxs_syr2k (upper) ---
  max error (upper): 0.00000E+00

--- libxs_syrk_task (OpenMP) ---
  threads=8 ntasks=32 libxs_syrk=1
  max error (omp):  0.00000E+00
  max error (omp tasks):  0.00000E+00

--- SYRK performance ---
  DSYRK:     <time> s (100 calls)
                 <rate> GFLOPS/s
  LIBXS:     <time> s (100 calls)
                 <rate> GFLOPS/s
  OMP:       <time> s (100 calls)
                 <rate> GFLOPS/s
  TASK:      <time> s (100 calls)
                 <rate> GFLOPS/s
```

Here `libxs_syrk=1`, i.e., the update is not split (see above).

Notes

- The dispatch step (libxs_syrk_dispatch / libxs_syr2k_dispatch) returns a pointer to a registry-owned config. This pointer remains valid until libxs_finalize or the registry is destroyed. There is no need to release it manually.
- Splitting is controlled by `LIBXS_GEMM_PANEL` (smallest column panel), `LIBXS_SYRK_BLAS` (BLAS SYRK for diagonal blocks, default on), and the tile sizes `LIBXS_GEMM_BM`, `LIBXS_GEMM_BN`, `LIBXS_GEMM_BK` (default 192x32x48); see `documentation/libxs_gemm.md`.
- Tasks need no synchronization. Scratch memory is thread-local and drawn from the LIBXS memory pool, so it appears in the pool statistics.

Metatoken Sample

The sample encodes the same input with each `libxs_tokenizer_t` granularity, prints the resulting logical metatokens, and verifies exact decoding.

Build

```
cd ../..
make PEDANTIC=2
make -C samples/tokenizer PEDANTIC=2
```

Building the library first is optional: without it, the sample compiles header-only.

Usage

```
./samples/tokenizer/tokenizer.x
./samples/tokenizer/tokenizer.x "token tokenization tokenizer token"
printf '%s\n' 'UTF-8 text here' | ./samples/tokenizer/tokenizer.x -
```

Without an argument the input is `Who is Alice? Alice saw 123.`; several arguments are joined by single spaces, and `-` reads stdin.

The input is encoded at the native, word and syllable granularity in turn. Each prints a line `NAME: input-bytes=N cells=N`, followed by one line per token with its cell offset, kind, logical byte length, physical cell count, and sentence-boundary flag. A nonzero exit status means encoding or round-trip verification failed.